

HaskellMOOC.fi Notes

JIANG AND SHARMA

<2026-05-19 Tue>

Contents

Abstract	3
1 Lecture 1: And so It Begins	4
1.1 Haskell Characteristics	4
1.2 Fundamental Concepts	4
1.3 Program Structure	4
2 Lecture 2: Recursion & Lists	5
2.1 Recursion & Control Flow	5
2.2 Lists & Polymorphism	5
2.3 Handling Optionality	5
3 Lecture 3: Higher-Order Functions	6
3.1 Functions as First-Class Citizens	6
3.2 Composition & Lambdas	6
3.3 Advanced List Patterns	6
4 Lecture 4: Type Classes & Folds	7
4.1 Tuples & Folds	7
4.2 Type Classes	7
5 Lecture 5: Algebraic Datatypes (ADTs)	8
5.1 Defining Custom Types	8
5.2 Recursive Types & Trees	8
5.3 Record Syntax	8
6 Lecture 6: Working Class Hero	9
6.1 Lecture Contents:	9

Abstract

Here we are documenting our learning in Haskell through Haskell-MOOC.fi and then some.

1 Lecture 1: And so It Begins

§1.1 Haskell Characteristics

- Functional, pure, and lazy
- Strongly typed with type inference
- Garbage collected and compiled
- Tooling: `stack ghci` (REPL), `:t` (type), `:load`, `:reload`

§1.2 Fundamental Concepts

- **Expressions:** Everything is an expression (no statements)
- **Application:** `f x y` (not `f(x,y)`); function calls bind tightest
- **Basic Types:** `Int`, `Integer`, `Double`, `Bool`, `String` (`[Char]`)
- **Currying:** `a -> b -> c` represents multi-argument functions

§1.3 Program Structure

- **Modules:** One per file
- **Definitions:** Optional type signature + equations
- **Logic:** `if...then...else` (else is mandatory), `where`/`=let`
- **Immutability:** Bindings are constant; “updates” create new values

2 Lecture 2: Recursion & Lists

§2.1 Recursion & Control Flow

- **Helper Functions:** Often marked with `'` (e.g., `fibonacci'`) to manage state
- **Tail Recursion:** Recursive call is the last action → optimizes to a loop
- **Guards:** `| condition = value` (first true guard wins)

§2.2 Lists & Polymorphism

- **Lists:** Homogeneous `[a]`, singly-linked, constructed via `cons` :
- **Common Ops:** `head`, `tail`, `take`, `drop`, `++`, `!!`, `reverse`
- **Polymorphism:** Lowercase type variables (e.g., `a`) determined by unification

§2.3 Handling Optionality

- **Maybe a:** `Nothing` | `Just x` (Safe alternative to null)
- **Either a b:** `Left a` | `Right b` (Commonly used for success/error)
- **Case expressions:** `case ... of` for complex pattern matching

3

Lecture 3: Higher-Order Functions

§3.1 Functions as First-Class Citizens

- **HOFs:** Functions that take or return functions (e.g., `map`, `filter`)
- **Partial Application:** `add 3` returns a function waiting for the second arg
- **Sections:** Partially applied operators, e.g., `(+1)`, `(*2)`

§3.2 Composition & Lambdas

- **Composition:** $(f \ . \ g) \ x = f \ (g \ x)$
- **Application Operator:** `$` (low precedence, avoids parentheses)
- **Lambdas:** Anonymous functions `\x -> x + 1`

§3.3 Advanced List Patterns

- **Comprehensions:** `[x | x <- xs, cond]`
- **Recursion Patterns:** Building via `:` and consuming via `x:xs`
- **Typed Holes:** Using `_` to let GHC suggest missing types

4 Lecture 4: Type Classes & Folds

§4.1 Tuples & Folds

- **Tuples:** Fixed-length, mixed types; accessed via `fst/=snd`
- **foldr:** “Crunches” a list into a single value
 - `foldr f z [] = z`
 - `foldr f z (x:xs) = f x (foldr f z xs)`

§4.2 Type Classes

- **Definition:** Groups of types sharing common operations (not OOP classes)
- **Constraints:** `(Eq a) => ...` restricts polymorphism to types that implement `Eq`
- **Key Classes:**
 - `Eq` ($\approx$, $\neq$) and `Ord` ($<$, $>$, `compare`)
 - `Num` $\rightarrow$ `Integral` / `Fractional` $\rightarrow$ `Floating`
 - `Show` (to string) and `Read` (from string)

5 Lecture 5: Algebraic Datatypes (ADTs)

§5.1 Defining Custom Types

- **ADTs:** Sum of constructors, product of fields (`data T = C1 | C2 Int String`)
- **Deriving:** Automatic implementation of `Show`, `Eq`, `Read`
- **Type Parameters:** Generic types, e.g., `data Maybe a`

§5.2 Recursive Types & Trees

- **Recursive Types:** Types that refer to themselves (e.g., custom `List a`)
- **Trees:** `Node val left right | Empty`
- **Operations:** Height calculation, BST lookup, and insertion

§5.3 Record Syntax

- Named fields instead of positional arguments
- Automatically generates accessor functions
- Improves readability and maintainability for large data structures

6 Lecture 6: Working Class Hero

§6.1 Lecture Contents:

- Defining new Type Classes
- Instantiating class using *instance* and *deriving* keywords